How FitPot Works

A plain-English tour of FitPot: every service it uses, every file in the repo, the core ideas behind them (each with an analogy), and how all the pieces talk to each other.

Read it top to bottom once; after that, use it as a reference. The companion doc BUILD_GUIDE.md covers *how to set things up*; this one covers *what things are and why*.

Contents

1. The big picture
2. Every service and tool
3. Core concepts, with analogies
4. Every file in the repo
5. How it all connects: step-by-step journeys
6. The data model
7. Security: who can do what
8. Glossary

---

1. The big picture

> **Analogy: FitPot is a small restaurant.**
>
> - **GitHub** is the recipe book: the master copy of every recipe (the code).
> - **GitHub Actions** is the kitchen: every time a recipe changes, it cooks a fresh batch.
> - **GitHub Pages** is the counter where the finished food is served (the website).
> - **Your phone's browser** is the customer's table, where the food is actually eaten (the app runs).
> - **Firebase Auth** is the doorman checking IDs.
> - **Firestore** is the order ledger everyone can see but no one can scribble in without permission.
> - **Security rules** are the house rules the ledger enforces on every write.
> - **Apps Script** is a trusted courier who carries photos to the storage room.
> - **Google Drive** is the storage room.

The key idea: there is no always-on server of our own. The app is a bundle of files that runs entirely inside each friend's browser. It talks directly to Google's services (Firestore, Auth) and to one small script (Apps Script) for photos. That's why the whole thing costs $0.

---

2. Every service and tool

Hosting and code

GitHub

- **What it is:** a website that stores code and its full history (every change, who made it, when).

- **In FitPot:** the `fitpot-grind` repo is the single source of truth. Everything else is built from it.
- **Analogy:** a shared Google Doc for code, with unlimited "version history".

Git / commits

- **What it is:** the version-control system GitHub is built on. A **commit** is a saved snapshot of the code with a message ("Phase 3: ledger…").
- **In FitPot:** each update you upload becomes a commit. If something breaks, any old commit can be restored.
- **Analogy:** save points in a video game.

GitHub Actions

- **What it is:** robots that run automatically when something happens in the repo.
- **In FitPot:** `.github/workflows/deploy.yml` tells a robot: "every time code lands on `main`, install everything, build the app, and publish it". Takes about a minute.
- **Analogy:** a kitchen that automatically re-cooks the dish every time the recipe changes.

GitHub Secrets

- **What it is:** a locked box in the repo settings for values the build needs but shouldn't be written into the code.
- **In FitPot:** the six `VITE_FIREBASE_*` values and `VITE_DRIVE_UPLOAD_URL`. The build robot reads them; nobody can view them again after saving.
- **Analogy:** a sealed envelope handed to the chef, not printed in the recipe book.

GitHub Pages

- **What it is:** free hosting for static websites (plain files: HTML, JavaScript, CSS, images).
- **In FitPot:** serves the built app at `kriticparikh.github.io/fitpot-grind/`. It only hands out files; it runs no code of its own.
- **Analogy:** a vending machine: it gives out exactly what's inside, never cooks anything.

The app itself

React

- **What it is:** a JavaScript library for building user interfaces out of **components** (reusable pieces like a button, a member row, a whole screen).
- **In FitPot:** every screen (Today, Calendar, Wall, Board, Group) is a React component. When data changes, React redraws only what changed.
- **Analogy:** LEGO bricks. Build small pieces once, snap them together into screens.

Vite

- **What it is:** the build tool. It takes the source code (many files, modern syntax) and bundles it into a few small, fast files browsers understand.
- **In FitPot:** `npm run dev` runs it locally; `npm run build` produces the `dist/` folder that gets published.
- **Analogy:** a packing service that takes your messy room of stuff and ships it in a few neatly labelled boxes.

PWA (Progressive Web App)

- **What it is:** a website that can be installed like an app: own icon, full screen, works offline for the shell.
- **In FitPot:** `vite-plugin-pwa` adds a **manifest** (name, icon, colours) and a **service worker** (caches the app's files and auto-updates them). "Add to Home Screen" uses these.
- **Analogy:** a website that's been given its own front door on your home screen.

npm and `package.json`

- **What it is:** npm is the package manager for JavaScript; `package.json` lists the libraries FitPot depends on.
- **In FitPot:** `firebase`, `react`, `react-router-dom`, `vite`, `vite-plugin-pwa`, `oxlint`.
- **Analogy:** a shopping list; `npm install` goes shopping.

React Router

- **What it is:** switches between screens based on the URL without reloading the page.
- **In FitPot:** `#/`, `#/calendar`, `#/wall`, `#/board`, `#/group`. The `#` (hash routing) is needed because GitHub Pages can't handle app-style URLs on its own.
- **Analogy:** tabs in a binder; flipping tabs doesn't mean fetching a new binder.

Google / Firebase

Firebase

- **What it is:** Google's toolkit of ready-made backend services for apps. You don't run servers; Google does.
- **In FitPot:** we use two Firebase products: **Authentication** and **Cloud Firestore**. Project name: `fitpot-grind`, on the free **Spark** plan (no card, can't be billed).
- **Analogy:** renting a fully-staffed office building instead of building your own.

Firebase Authentication (Auth)

- **What it is:** handles sign-in. FitPot offers **Continue with Google** and, since phase 5.3, **email + password** (with verification emails and password-reset emails that Firebase sends for free).

- **In FitPot:** after sign-in, each person gets a permanent **uid** (user ID) and a short-lived **ID token** that proves who they are to other services.
- **Analogy:** the doorman who checks your ID once, then gives you a wristband (the token) that other staff recognise.

Cloud Firestore (the database)

- **What it is:** a NoSQL database in Google's cloud. Data is stored as **documents** (like small JSON records) inside **collections** (folders of documents). Supports **live listeners**: apps get changes pushed instantly.
- **In FitPot:** stores groups, members, check-ins, rest-day votes, penalty/target changes and photo records. When a friend checks in, your screen updates within a second without refreshing.
- **Analogy:** a shared notebook in the cloud. Every page is a document; every section is a collection; everyone watching sees new writing appear live.

Firestore Security Rules

- **What it is:** a small rulebook (`firestore.rules`) that Firestore checks **on every single read and write**, on Google's side.
- **In FitPot:** "only members can see the group", "you can only check in as yourself", "a check-in needs a photo if the group requires one", "check-ins are stamped with server time"…
- **Why it matters:** the app runs on friends' phones, so a clever friend could send any request they like. The rules are what actually protect the data; the app's buttons are just convenience.
- **Analogy:** a bouncer who reads the house rules for every single person at every single door, no matter what the app's UI says.

Google Apps Script

- **What it is:** free JavaScript that runs on Google's servers, attached to your Google account. It can use Drive, Gmail, Sheets, etc. as **you**. Deployed as a **web app**, it gets a URL (`…/exec`) that can receive requests.
- **In FitPot:** `apps-script/Code.gs` is our only "backend". It receives a photo, checks the sender is a real signed-in group member, and saves it into the organiser's Drive. It also serves photos back to members.
- **Script properties:** private settings for the script (API key, project ID, timezone), kept out of the code.
- **Analogy:** a trusted courier with a key to your storage room, who only accepts packages from people wearing a valid wristband.

Google Drive

- **What it is:** Google's file storage (15 GB free, shared with Gmail).
- **In FitPot:** stores workout photos at `FitPot/<member>/<YYYY-MM>/<date_time>.jpg`, private to the organiser's account.
- **Analogy:** the storage room. Only the courier (Apps Script) goes in and out.

Identity Toolkit API / Firestore REST API

- **What they are:** Google web addresses that Apps Script calls behind the scenes.
- **In FitPot:** Identity Toolkit answers "whose ID token is this?"; the Firestore REST API lets Apps Script read the caller's member record *using the caller's own token*, so the security rules decide whether they're really in the group.
- **Analogy:** the courier phoning the doorman to double-check a wristband.

Food databases (phase 5)

- **Indian Nutrient Databank (INDB):** 1,014 common Indian dishes with nutrition per 100 g and per serving, from Indian food composition tables and published research. FitPot ships a compact copy inside the app (`public/data/indb.json`), so Indian search is instant and works offline.
- **USDA FoodData Central:** the US government's food database: generic foods, US brands, mixed dishes. Searched live with a free key.
- **Open Food Facts:** a free, crowd-sourced database of packaged products worldwide, looked up by name or barcode.
- None of these are AI; they're lookup tables.
- **Analogy:** three reference books on the kitchen shelf: an Indian cookbook, an American one, and a catalogue of every packet in the supermarket.

Barcode scanner (ZXing)

- **What it is:** open-source barcode-reading code. Android Chrome has a built-in reader; on other phones FitPot loads ZXing, which runs on the phone itself.
- **In FitPot:** reads the EAN/UPC number printed on packs, then looks it up in Open Food Facts (or your own foods).
- **Analogy:** the supermarket's checkout scanner, pointed at your own pantry.

Google Identity Services + OAuth ("Connect my Google Drive")

- **What it is:** Google's official way for an app to ask *you* for permission to use part of *your* Google account, and receive a short-lived **access token**.
- **In FitPot:** used only for progress photos, with the `drive.file` permission: FitPot can see and manage only the files it created in your Drive, nothing else.

- **Analogy:** a hotel key card that opens one room (the FitPot Progress folder), not the whole hotel, and stops working after an hour.

Google Drive API

- **What it is:** the web address apps use to create, read and delete files in someone's Drive.
- **In FitPot:** progress photos are uploaded straight from your phone into **your own** Drive (`FitPot Progress/`) using the token above. They never pass through the organiser's script or Drive.

Exchange rates: Frankfurter (phase 5.1)

- **What it is:** a free, open web service that publishes the European Central Bank's daily exchange rates. No key, no account.
- **In FitPot:** used only when someone switches the pot's currency: FitPot asks "1 ₹ = how many $ today?" and pre-fills the answer (editable). If it's unreachable, a second free source (the open `currency-api` files on jsDelivr) is tried. The rate is then saved in the group's log, so it's never looked up again.
- **Analogy:** the exchange-rate board at the airport: you glance at it once when you swap money, then keep the receipt.

Coming later

- **Google Colab** (phases 6–7): free notebooks with GPUs for training the vision and language models.
- **Hugging Face Hub** / **TensorFlow.js** / **MediaPipe** (phases 6–7): store trained models and run them in the browser.
- **Firebase Cloud Messaging** (phase 7): push notifications for nudges.

---

3. Core concepts, with analogies

Frontend vs backend vs database

- **Frontend:** what runs on the phone and what you see: the screens, buttons, camera. (React app.)
- **Backend:** code running on a server you trust, for jobs the phone shouldn't be trusted with. (Apps Script for photos; the security rules act as a mini-backend for data.)
- **Database:** the permanent memory. (Firestore.)
- **Analogy:** a restaurant: the dining room (frontend), the kitchen (backend), the pantry (database).

Static site

The website is just files. GitHub Pages doesn't compute anything per visitor; the phone does all the work. **Analogy:** a printed menu vs a waiter: GitHub Pages is the printed menu.

Build step

Source code (`src/`) isn't what the browser runs. Vite **builds** it into `dist/` (compressed, bundled). GitHub Actions does this on every push. **Analogy:** a manuscript (source) vs the printed book (build).

Environment variables (`VITE_*`)

Values baked into the app at build time (Firebase config, Drive URL). Locally they live in `.env.local`; on GitHub, in Secrets. **Analogy:** filling in the blanks of a form letter just before it's printed.

Authentication vs authorization

- **Authentication** = *who are you?* (Firebase Auth, Google sign-in.)
- **Authorization** = *what are you allowed to do?* (Firestore rules, Apps Script membership check.) **Analogy:** showing your ID at the door (authentication) vs your ticket only letting you into certain rooms (authorization).

ID token

A signed, short-lived (≈1 hour, auto-refreshed) proof of identity from Firebase. Sent with requests so others can verify you without your password. **Analogy:** a festival wristband: hard to forge, expires, and staff can check it.

Real-time listeners (`onSnapshot`)

The app doesn't keep asking "anything new?". It subscribes once, and Firestore pushes changes. **Analogy:** following someone on social media vs checking their profile every minute.

Server timestamp

When the app writes `serverTimestamp()`, Google fills in the time on *its* clock, and the rules check it equals `request.time`. The phone's clock is never trusted. **Why:** otherwise someone could set their phone to yesterday and backdate a check-in. **Analogy:** a post office postmark vs the date you wrote on the letter yourself.

Your own timezone and "a day" (since phase 5.1)

Every member's day follows **their own phone's timezone**, saved on their member record when they join. Your check-in, photo, undo and rest vote all belong to **your** local date, midnight to midnight where you are. (Before 5.1 the whole group shared the creator's timezone; older

members still use it until their phone reports a different one.) **Analogy:** each friend wears their own watch, and each watch is honest because Google's server time is read through it.

The group's day closes after the last midnight

A day counts toward the pot only once it has ended for **everyone**: after the midnight of the member furthest behind (furthest west). FitPot calls this the **closed-through** day. Weekly groups settle once Sunday has ended for everyone. **Example:** New York and India. At 10 PM Friday in New York it's 7:30 AM Saturday in India. Friday is over in India, but the group's Friday closes at New York's midnight. **Analogy:** a relay race across time zones: the day's results are posted only when the last runner crosses the line.

Timezone changes start tomorrow

If your phone's timezone changes (you fly from Raleigh to Bengaluru), the app logs it in `tzchanges` and it takes effect from **your next day**. Otherwise someone could switch the setting to stretch the day they're in by hours. Each log entry is stamped with server time, and the ledger ignores any change that wasn't made before the day it starts. **Analogy:** changing your watch when you land, but only after today's results are posted.

Settings start after the furthest-ahead member's today

Penalty, target and currency changes start the day after the **furthest-ahead** member's today. A friend in India whose Saturday has already begun never sees Saturday's stakes change mid-day.

One pot currency, converted when it changes

Each group has one currency at a time: ₹ INR, $ USD, £ GBP or C$ CAD. Switching logs **the rate on the day** (1 old = rate new) and the new penalty; both start tomorrow. From then on **every** amount, including old months, is shown in the new currency: an old ₹ penalty × the saved rate. Several switches multiply their rates (₹ → $ → £). Nothing in the database is rewritten; like balances, the conversion is recalculated from the log. **Analogy:** converting your old travel receipts at the rate printed on the day you changed money, then keeping one set of books.

Batched writes

Several database changes that succeed or fail **together** (e.g. photo record + check-in; group + passcode + first member). **Analogy:** a bank transfer: money leaves one account only if it arrives in the other.

Derived ledger (no stored balances)

FitPot never stores "Asha has +$12". Balances are **recomputed** every time from the raw facts: check-ins, members, rest votes, penalty, target, timezone and currency changes. Nobody can

edit a balance because there is no balance to edit. **Analogy:** a scoreboard calculated from the match footage, not a number someone types in.

Daily vs weekly settling

- **Every day (7):** each day, those who missed pay the penalty; the pot splits among those who checked in.
- **1–6 days a week:** at Sunday midnight, anyone under the target pays the penalty × days short; the pot splits among everyone who hit it. Each group rest day lowers the target by 1. **Analogy:** a daily fine at the door vs a weekly attendance report card.

Change logs that start "next time"

Penalty and target changes are **appended to a log** with an effective date (tomorrow / next Monday). Old days and the current week never change. **Analogy:** a gym raising membership fees from next month, not retroactively.

Settle-up (fewest transfers)

Given everyone's net balance, a greedy algorithm pairs the biggest debtor with the biggest creditor, repeatedly, to minimise the number of payments. **Analogy:** at dinner, instead of everyone paying everyone, one friend pays another the difference.

Camera-only capture

The app uses the browser's live camera API (`getUserMedia`) and never shows a file picker, so there's no way to choose an old gallery photo. The watermark is drawn onto the image before upload. **Analogy:** a photo booth: you can only get a photo by standing in it now.

Caching

Photos downloaded once are saved on the phone (Cache Storage) so the Wall doesn't re-download them. **Analogy:** keeping a copy of a book at home instead of going back to the library each time.

Private vs group data

Group data (check-ins, photos on the Wall, rest votes, the pot) lives under `groups/…` and every member can read it. Personal data (meals, own foods, targets, weight, progress-photo records) lives under `private/{your uid}/…`, and the rules allow **only you**. **Analogy:** the team noticeboard vs your own diary with a lock.

Per-100 g nutrition and units

Database foods store nutrition per **100 g**. Each unit (g, bowl, plate, serving) knows how many "100 g's" it is, e.g. a 292 g bowl = 2.92. Your amount × that factor × the per-100 g numbers = what you ate. Your own foods store nutrition **per serving** instead. **Analogy:** a recipe that says "per 100 g"; the unit is just a measuring cup with its size written on it.

Recipes

A recipe adds up its ingredients' nutrition, then divides by the number of servings. It's saved as one of your foods, so logging it is one tap. **Analogy:** a meal-prep box: weigh everything once, split into portions, label each portion.

Me first, groups optional (phase 5.2)

After sign-in you land on **Me**, your private area. Groups are something you add: Today, Calendar, Wall and Board need one, Me never does. **Analogy:** your own gym notebook comes first; the friends' group chat is something you join.

Several groups, one check-in

Your profile keeps a list of your groups (`groupIds`) and the one you're viewing (`groupId`). One check-in, or one workout photo, is written into **every** group you're in, each in its own batch so one group's problem never blocks the others. Each group gets its own copy of the photo, because the Drive script only shows a photo to members of the group it was uploaded for. **Analogy:** one gym badge scan that marks you present on every team roster you're on.

Leaving a group

Leave stamps `leftAt` (server time) on your member record, once. You stop counting from your next day; today and a week you've started still count. Your past results stay, so balances never change after the fact. The rules stop someone who has left from checking in or changing settings there. **Analogy:** handing back your locker key. Your name stays on last season's scoreboard.

Progressive overload

Getting stronger means slowly asking more of a muscle: one more rep, or a bit more weight, than last time. FitPot shows **last time** for every exercise and a **beat it** target (double progression: add reps until 12, then add weight and drop to 8). **Analogy:** a video game level you replay with the previous high score on screen.

Estimated 1-rep max and personal records

Comparing "60 kg × 10" with "65 kg × 5" needs one number. The **Epley formula**, weight × (1 + reps ÷ 30), estimates the most you could lift once. FitPot walks your workouts in date order and marks a **PR** when you beat your heaviest weight, best estimated 1-rep max, or most reps. Your first session of an exercise sets the baseline. **Analogy:** converting different race distances into one pace so they can be compared.

Sets and volume per muscle, per week

Every exercise belongs to a muscle group. Each week (Mon–Sun) FitPot totals **sets** and **volume** (weight × reps) per muscle and compares each exercise's best set with the week

before. Like the pot, it's recomputed from your raw workouts, never stored.

Email verification (phase 5.3)

A new email account gets a link to prove the address is real. Until it's clicked, the account works for Me but can't join or create groups; the rules check `email_verified` on the sign-in token. Google accounts are verified by Google. After clicking, the app refreshes the token so the rules see the change. **Analogy:** a gym membership that starts once you confirm your phone number.

One account, two keys

Firebase keeps one account per email. A Google account can **add a password** ("linking"), so the same account opens with either key. Sensitive changes (new password, delete account) ask you to sign in again first (re-authentication). **Analogy:** one house, a front-door key and a back-door key; the locksmith asks for ID before changing a lock.

Deleting an account

FitPot deletes your private Me data and your login, leaves your groups and renames you "Former member" there. Group history stays, because friends' balances are built from it. **Analogy:** leaving a club: your locker is emptied, but the old match results still show "a former player".

Two different photo paths

- **Workout photos** (group proof): phone → organiser's Apps Script → organiser's Drive; everyone in the group can see them.
- **Progress photos** (private): phone → *your own* Drive, directly, with your own permission. Nobody else can see them. **Analogy:** posting on the team noticeboard vs putting a photo in your own locker.

Access token vs ID token

- **ID token** (Firebase): proves *who you are* to FitPot's database and script.
- **Access token** (Google OAuth): lets FitPot *act in your Drive*, only within `drive.file`, for about an hour. **Analogy:** your ID badge vs a one-room key card.

Linting

`oxlint` reads the code for common mistakes before it ships. **Analogy:** spell-check for code.

---

4. Every file in the repo

Top level

File	What it is	What it does
`README.md`	Front page of the repo	Short intro, stack, roadmap, links to the guides.
`package.json`	Project manifest	Names the app, lists libraries (dependencies) and commands (`dev`, `build`, `lint`).
`package-lock.json`	Exact versions	Pins every library's exact version so every build is identical. Never edit by hand.
`index.html`	The one HTML page	The empty shell the browser loads; `<div id="root">` is where React draws everything. Also loads the fonts and sets the theme colour.
`vite.config.js`	Build settings	Tells Vite to use React, sets the base path (`/fitpot-grind/`) for GitHub Pages, and configures the PWA (name, icon, colours, auto-update).
`firestore.rules`	Database security rules	The rulebook Firestore enforces on every read/write. Pasted into the Firebase console. See section 7.
`.env.example`	Template for settings	Lists the `VITE_*` values the app needs. Copy to `.env.local` for local development; on GitHub they're Secrets.
`.gitignore`	Ignore list	Files Git should never store: `node_modules`, `dist`, `.env.local` (your private values).
`.oxlintrc.json`	Linter settings	Which code checks `oxlint` runs (e.g. React hooks rules).

`.github/workflows/`

File	What it does
`deploy.yml`	The GitHub Actions recipe: on every push to `main` → check out code → set up Node.js → `npm install` → `npm run build` (with Secrets as environment variables) → upload `dist/` → publish to GitHub Pages.

`public/`

File	What it does
`icon.svg`	The app icon (lime ring, orange dashes, `$`). Used for the browser tab and the home-screen icon. Copied as-is into the build.
`data/indb.json`	The Indian Nutrient Databank, compacted: code, name, kcal/protein/carbs/fat/fibre per 100 g, and a typical serving (unit + grams). Loaded on first Indian search, then cached for offline use.

`apps-script/`

File	What it does
`Code.gs`	The photo backend, pasted into script.google.com. `doPost` handles requests: verifies the Firebase ID token (Identity Toolkit), checks group membership through Firestore using the caller's token, then either **uploads** (saves the JPEG to `FitPot/<name>/<month>/`, tags it with the group ID) or **gets** (returns a photo only if it belongs to the caller's group). `doGet` is the health check (`"configured":true`).

`docs/`

File	What it does
`BUILD_GUIDE.md`	Step-by-step setup and deploy instructions for every phase, with fixes.
`HOW_IT_WORKS.md`	This file.

`ml/`

File	What it does
`README.md`	Plan for the two learning tracks (vision and language models), filled in during phases 6–7.

`src/` — the app

Entry points

File	What it does
`main.jsx`	The very first code that runs. Finds `<div id="root">` in `index.html` and tells React to draw `<App />` there. Loads `index.css`.
`App.jsx`	The traffic controller. **Setup notice** (Firebase not configured) → **Login** (not signed in) → the **Shell** (top bar with the group switcher, the tabs, bottom navigation), landing on **Me**. `GroupProvider` loads the group you're viewing with its timezone and currency logs and attaches `group.clock` and `group.money`; `NeedGroup` shows "Join or create a group" on group tabs until you're in one. Also watches your phone's timezone to log a change.
`groupContext.js`	Shares the group you're viewing (status, group, members) with the screens that need it.
`firebase.js`	Connects to Firebase using the `VITE_FIREBASE_*` values. Exports `auth` (sign-in), `db` (Firestore) and the Google sign-in provider. If the values are missing, `isConfigured` is false and the app shows the setup notice.
`index.css`	All styling: the dark theme, lime/orange colours, fonts, cards, buttons, calendar grid, camera overlay, photo grid. Colours are defined once as variables (`--lime`, `--orange`…) and reused.
`hooks.js`	**Live data subscriptions.** Small functions each screen calls to get data that updates itself: `useUserDoc` (your profile), `useGroup`, `useMembers`, `useCheckins` (by date range), `useMyCheckins` (for streaks), `useSubcollection` (rest votes, penalty/target logs), `useDayDocs` (e.g. today's photos), `useGroupsMeta` (names and settings of all your groups, for the switcher). Each opens a Firestore listener and closes it when the screen goes away.

`src/lib/` — logic (no screens)

File	What it does
`dates.js`	Date maths on `YYYY-MM-DD` strings: the day in a given timezone, add days, days in a month, weekday, Monday of a week, streaks (current and best).
`clock.js`	**Each member's own clock, pure logic** (phase 5.1). `makeClock` knows every member's timezone and logged changes and answers: your today, a friend's today, which timezone someone used on a given date, whether a day is still open for them, the furthest-ahead day, and `closedThrough` (the last day that has ended for everyone). Also `validChanges` (drops setting changes made too late) and `onItsDay` (a check-in/photo counts only if server time falls on its date in that person's timezone).
`money.js`	**Pot currency** (phase 5.1). The four currencies, formatting (`₹1,234.50`, `C$5.00`), today's exchange rate from Frankfurter (with a fallback), and `makeMoney`: the currency on any day and `factorOn(day)`, the multiplier that converts that day's amounts into today's currency.
`group.js`	**Actions that write to the database:** create a group (generates the passcode, stores target/photo rule/currency), join with a passcode (saves your phone's timezone), check in, undo a check-in, vote rest day, change penalty, change weekly target (from next Monday), change currency (with rate and new penalty), log a timezone change, toggle photo requirement. Also `joinDay` (from server time, in your timezone).
`ledger.js`	**The money engine, pure maths.** `computeLedger` turns raw facts into daily/weekly results and each member's net total, following the group's daily or weekly rule, with every penalty converted into today's currency. `settleUp` works out who pays whom with the fewest transfers. `penaltyOn` / `targetOn` read the change logs. Has no Firebase code, so it can be tested on its own.
`useLedger.js`	The bridge between the database and the money engine. `useLedgerInputs` loads check-ins, rest votes and change logs and cleans them up using each member's own timezone (ignores invalid entries); `runLedger` feeds them into `computeLedger` with the group's settings.
`foods.js`	**Food search.** Searches 🇮🇳 INDB (from the bundled file), 🇺🇸 USDA and 📦 Open Food Facts, plus ⭐ your own foods, and turns every result into the same shape (nutrition + units). Also barcode lookup, nutrient scaling and totals.

File	What it does
`privateData.js`	**Your private data.** Live hooks and writes for meals, own foods and recipes, targets and settings, weight entries, progress-photo records, and (phase 5.2) training setup, workouts, your own exercises, routines and body measurements, all under `private/{you}/`. `readAllPrivate` reads everything once for the export.
`myDrive.js`	**Your own Google Drive.** Loads Google's sign-in library, asks for the `drive.file` permission, creates the `FitPot Progress` folder, uploads, shows and deletes progress photos.
`image.js`	Shrinks and compresses a picture before upload.
`account.js`	**Sign-in and account** (phase 5.3): Google and email sign-in, create account, reset password, verification, `isVerified`, change name / password, add a password to a Google account, re-authentication, delete account. Also turns Firebase error codes into plain English.
`everywhere.js`	**One check-in for all your groups** (phase 5.2). `myDayInGroups` works out your day in each group (and tells a group if your timezone changed); `checkInEverywhere` uploads the photo per group and writes each group's photo + check-in; `undoEverywhere` undoes today everywhere.
`training.js`	**Training maths, pure logic** (phase 5.2). Muscle groups, equipment, the built-in exercise library, `e1rm` (Epley), `lastTime`, `beatIt`, `personalRecords`, `weekByMuscle` (sets, volume and best set vs last week), kg/lb display helpers. No Firebase, so it's testable on its own.
`photos.js`	Talks to the Apps Script: `uploadToDrive` (one group), `uploadPhoto` (send the photo, then record the photo, and the check-in if required, in one batch), `photoUrl` (fetch a photo and cache it on the phone), `isValidPhoto`. `photosEnabled` is true only if the Drive URL secret was set.

`src/components/` — reusable pieces

File	What it does
`Avatar.jsx`	A member's round Google profile photo, or their initials if there isn't one.
`VerifyBanner.jsx`	"Confirm your email" card with **I've confirmed** and **Resend email**; shown in Me, Account and instead of join/create until verified.
`JoinCreate.jsx`	Join a group with a passcode, or create one (name, target, currency, penalty, photo rule). Shown in the Group tab. (Was `screens/Onboarding.jsx`.)
`WorkoutEditor.jsx`	Log or edit a workout: exercises → sets (weight × reps), last time + beat-it target per exercise, ✓ starts the rest timer, notes, and the 🏆 PR screen on save.
`ExercisePicker.jsx`	Exercise finder: search, muscle-group chips, "only what my equipment allows", create / delete your own exercises.
`RestTimer.jsx`	Countdown bar after a set, +15 s / Skip, vibrates when done (Android).
`Measurements.jsx`	Body measurements by date (cm or in), latest vs first, all entries.
`ExportData.jsx`	Export my data: CSV per area, or everything as JSON.
`BarcodeScanner.jsx`	Live camera view that reads a pack's barcode (or lets you type the number).
`FoodPicker.jsx`	The food finder: search box, source chips (🇮🇳 🇺🇸 📦), Recent, My foods, barcode button, results list.
`PortionPicker.jsx`	Choose amount × unit and the meal, see the nutrition live, add it, or ⭐ save it to My foods.
`MyFoodForms.jsx`	**Create food** (your own item, per serving, optional barcode) and **Recipe builder** (ingredients → per-serving nutrition).
`WeightChart.jsx`	The weight trend line with a goal line and tap/hover tooltip.
`CameraCapture.jsx`	The full-screen camera: one-time consent screen → opens the live camera (front/back **Flip**) → **capture** → draws the `FitPot · name · date/time` watermark → shrinks to ≤1080 px and compresses to ~300–500 KB → **Retake** or **Check in + post** / **Post to group** → calls `uploadPhoto`. No file picker exists anywhere.

`src/screens/` — one file per screen

File	Screen	What it does
`Today.jsx`	**Today** tab	The big CHECK IN button (opens the camera when a photo is required), Undo check-in, this week's progress (weekly groups) or today's stakes (daily groups), your month balance, rest-day voting, the group's rule, and the crew's live status.
`Calendar.jsx`	**Calendar** tab	Month grid coloured by your status (done / missed / rest / today), streak stats, and a day breakdown when you tap a date.
`Wall.jsx`	**Wall** tab	The group's workout photos for a chosen day, plus the button to post (or check in with) a photo.
`Board.jsx`	**Board** tab	Month balances (with the shame 🙈), settle-up list, weekly results, streak leaderboard, and totals.
`Login.jsx`	Login	Continue with Google, or email + password: Sign in · Create account · Forgot password.
`Account.jsx`	⚙️ **Account**	Name, sign-in methods, verification, change or add a password, sign out, delete my account.
`Me.jsx`	**Me** tab (where you land)	Private area with four sections: Food, Training, Progress, Targets (the Targets form, body measurements and data export live here). Works without a group.
`Training.jsx`	Me → Training	Log (start / repeat / routine, history with PRs), Muscles (weekly progressive overload), Records (per exercise + 1-rep max chart), Routines, Setup (goal, days, length, experience, rest timer, equipment, limitations).
`FoodLog.jsx`	Me → Food	Daily diary by meal, totals vs targets, add/remove entries, My foods & recipes list.
`Progress.jsx`	Me → Progress	Manual weight log, trend chart, 7/30-day change, entry list, and optional progress photos in your own Drive with side-by-side compare.
`Group.jsx`	**Group** tab	Your groups (switch), join or create another, then for the group you're viewing: passcode + invite, **Group rules** (weekly target, photo requirement, penalty), **Pot currency** (switch with today's rate), members with their timezone and local time, Leave this group, disclaimer.

Generated / not in the repo

Folder	What it is
`node_modules/`	Downloaded libraries (created by `npm install`). Huge; never committed.
`dist/`	The built website (created by `npm run build`). GitHub Actions builds and publishes it; never committed.
`.env.local`	Your local Firebase values for `npm run dev`. Private; never committed.

---

5. How it all connects: step-by-step journeys

A. You push an update

1. You upload files on GitHub → a new **commit** on `main`.
2. **GitHub Actions** wakes up (`deploy.yml`): installs libraries, runs **Vite** to build `dist/`, injecting the **Secrets** as `VITE_*` values.
3. `dist/` is published to **GitHub Pages**.
4. Next time a phone opens the app, the **service worker** notices a new version and swaps it in (sometimes needs one more open).

B. A friend opens the app

1. The browser downloads `index.html` + the bundled JavaScript from **GitHub Pages**.
2. `main.jsx` starts React → `App.jsx` runs.
3. `firebase.js` connects to **Firebase** with the baked-in config.
4. **Auth** reports whether they're signed in. If not → Login screen → **Google sign-in** popup → Auth issues a **uid** and **ID token**.
5. `hooks.js` subscribes to their profile in **Firestore** → the Shell opens on **Me**. If they're in groups, the group they last viewed loads for the group tabs; if not, those tabs offer "Join or create a group".

C. Creating and joining a group

1. **Create:** `group.js` picks a random passcode, then writes **in one batch**: the group, a `passcodes/{code}` lookup, the creator's member record (with server `joinedAt`), and the creator's profile `groupId`. The **rules** check the creator is who they say, the passcode lookup matches the group, and `joinedAt` is server time.
2. **Join:** `group.js` looks up `passcodes/{code}` → gets the group ID → writes the member record (rules verify the passcode really belongs to that group) + profile `groupId`.

D. Checking in (photo required) — see also P for several groups

1. Tap 📸 **CHECK IN** on Today → `CameraCapture` opens the live camera (first time: consent screen).
2. Snap → watermark drawn → compressed JPEG.
3. **Check in + post** → `photos.js` gets the user's **ID token** from Auth and sends `{photo, token, groupId}` to **Apps Script**.
4. **Apps Script**: asks **Identity Toolkit** who the token belongs to → reads the member record from **Firestore** *as that user* (the rules decide) → saves the JPEG into **Google Drive** `FitPot/<name>/<month>/` → returns the Drive file ID.
5. Back on the phone, `photos.js` writes **one batch** to Firestore: the photo record + the check-in, both with **server timestamps**.
6. **Firestore rules** check: you're a member, it's your own uid, the doc ID is `<date>_<you>`, time is server time, and, since the group requires a photo, the photo record exists in the same batch.
7. Every friend's app has a **live listener** on today's check-ins and photos → their Today and Wall update within a second.

E. Undo a check-in

`group.js` deletes today's check-in and photo record in one batch. Rules allow deleting **only your own, within 24 hours**. The JPEG stays in Drive as a record.

F. Viewing the Wall

1. `hooks.js` listens to `photos` where `date == the chosen day`.
2. For each, `photos.js` asks **Apps Script** for the image (with the viewer's token). Apps Script checks membership and that the photo's stored group ID matches, then returns it.
3. The image is saved in the phone's **cache**, so it downloads only once.

G. The week settles (weekly groups)

Nothing "runs" at Sunday midnight. Instead, whenever someone opens Today or Board:

1. `useLedger.js` loads the raw facts from **Firestore**: check-ins, rest votes, penalty and target change logs.
2. It drops anything invalid (e.g. a check-in whose server time doesn't match its date; a change made on or after its effective date).
3. `ledger.js` walks each Monday→Sunday week that has **finished**: counts each member's workouts, lowers the target by the rest days, finds who's short, builds the pot and splits it among those who hit the target.
4. Board shows the totals and `settleUp` turns them into "Ben pays Asha $10".

Because it's recomputed from facts, everyone's phone gets the exact same answer.

H. Voting a rest day

Tap **Vote rest** → writes your vote into `restdays/{today}` with server time. Rules allow you to add or remove **only your own** vote. The ledger counts a day as a rest day if more than half the active members voted **on or before** that day.

I. Logging food

1. **Me → Food → + Add** under a meal → `FoodPicker` opens and shows **Recent** and **My foods** straight away (read from `private/{you}/…`).
2. Typing (e.g. "paneer") waits a moment, then searches in parallel: `foods.js` filters the bundled 🇮🇳 INDB file on the phone, and calls 🇺🇸 USDA and 📦 Open Food Facts over the internet. Each source fills in as it answers; if one fails, the others still show.
3. Tap a result → `PortionPicker`: choose amount × unit (e.g. 1 bowl = 202 g) → nutrition updates live → **Add to Lunch**.
4. `privateData.js` writes a meal entry to `private/{you}/meals` (date, meal, amount, nutrition, and a copy of the food so it appears in Recent). The rules check it's your own area.
5. The diary's live listener updates the totals and bars instantly.

J. Scanning a barcode

1. ▦ **Scan barcode** → `BarcodeScanner` opens the back camera and reads the number several times a second.
2. FitPot first checks **your own foods** for that barcode, then Open Food Facts.
3. Found → Portion screen. Not found → **Create food** opens with the barcode filled in; you type the label once, and next time it's found instantly in your own foods.

K. Building a recipe

My foods & recipes → 🍲 **Recipe** → name + servings → **+ Add ingredient** (the same finder and portion screens) → FitPot adds up all ingredients and divides by servings → **Save.** It's now a ⭐ food you can log in one tap.

L. Logging weight and progress photos

1. **Me → Progress** → type your weight, pick the date → saved to `private/{you}/weights/{date}` in kg (converted if you use lb). The chart redraws.
2. **Connect my Google Drive** → Google's popup asks you to allow FitPot to use *files it creates*. You get a one-hour access token.
3. **+ Add progress photo** → choose camera or gallery → `image.js` compresses it → `myDrive.js` uploads it to **your** Drive's `FitPot Progress` folder → FitPot stores only the

file ID and date in `private/{you}/progress`.

4. Viewing: FitPot asks your Drive for each file with your token. Nobody else has a token for your Drive, so nobody else can see them.

M. A friend in another country (phase 5.1)

1. Asha joins from India → her member record saves `timezone: Asia/Kolkata`.
2. At 7:30 AM IST Saturday (10 PM Friday in New York) she checks in → the check-in is dated **Saturday**, her day. The ledger accepts it because Google's server time, read in IST, is Saturday.
3. On your Today (still Friday), Asha shows **Missed** or **In** ✓ for Friday, and "7:30 AM Sat there".
4. At your midnight, Friday has ended for everyone → `closedThrough` becomes Friday → Board settles it.

N. Travelling

1. You fly from New York to India and open the app.
2. `App.jsx` sees the phone says `Asia/Kolkata` but FitPot has you on New York → writes `tzchanges` {you, Asia/Kolkata, effective: your tomorrow}.
3. Today shows "switching to IST from tomorrow". The rest of your current day stays on New York time.

O. Switching the pot to dollars

1. **Group → Pot currency → $ US dollar** → `money.js` asks Frankfurter for today's ₹→$ rate → pre-fills it and the penalty (₹100 × 0.0118 ≈ $1.18).
2. **Switch** → one batch: a `currencies` entry {from ₹, to $, rate, penalty, effective tomorrow} + a `penalties` entry.
3. From tomorrow, `factorOn` multiplies every older ₹ amount by the rate, so the Board's months, settle-up and Today all show $.

P. One check-in, two groups (phase 5.2)

1. You're in *Morning Grinders* ($, weekly) and *Office Lifters* (₹, daily). Either requires a photo → 📸 **CHECK IN** opens the camera.
2. **Check in + post** → `everywhere.js` reads, for each group, your member record and timezone log → your date there → whether you're already in.
3. For each group: upload the photo to Drive for that group → one batch with that group's photo record + check-in.
4. Both groups' Today and Wall update live. **Undo** reverses both.

Q. Leaving a group

Group → Leave this group → one batch: your member record gets `leftAt` (server time) and the group is removed from your `groupIds`. From tomorrow the ledger stops counting you; your history stays.

S. Signing up with email (phase 5.3)

1. Login → **Create account** → name, email, password → `account.js` creates the Firebase user, saves the name, and asks Firebase to email a verification link.
2. You land on **Me** with the 📧 banner. The Group tab shows the same banner instead of join/create.
3. Tap the link in the email → back in FitPot → **I've confirmed** → the app reloads your account and gets a fresh token with `email_verified: true` → join/create appears, and the rules now allow it.

T. Deleting an account

⚙️ → **Delete my account…** → confirm with password (or Google) → type DELETE → `account.js` deletes every document under `private/{you}`, marks you left and "Former member" in each group, deletes `users/{you}`, then deletes the login. Friends' Boards are unchanged.

R. Logging a workout and beating last week

1. **Me → Training → ▶ Push day** → the editor opens with the routine's exercises, weights pre-filled from last time.
2. Each exercise shows "Last time: 60 kg × 10, 62.5 kg × 8 · beat it: 60 kg × 11".
3. Tick ✓ after each set → rest timer runs.
4. **Save** → `privateData.js` writes `private/{you}/workouts/{id}` (kg) → `personalRecords` compares with every earlier session → 🏆 screen if you beat something.
5. **Muscles** recomputes the week: chest sets and volume vs last week, bench ▲.

---

6. The data model

Firestore is organised as **collections → documents → fields**. Paths read like folders.

```
users/{uid}                              your profile
  name, photoURL, groupId (the group you're viewing), groupIds (all your
groups), leftGroupIds, photoConsent

passcodes/{CODE}                         lookup table for joining
  groupId

groups/{groupId}                         one friend group
  name, passcode, penalty (base), currency (base), weeklyTarget (base),
photoRequired, timezone (fallback), createdBy, createdAt

  members/{uid}                          who's in the group
    name, photoURL, passcode, timezone (your phone's, locked), joinedAt (server
time), joinedDate, leftAt (when you left; set once)

  checkins/{YYYY-MM-DD_uid}              one per person per day
    uid, date, createdAt (server time)

  photos/{YYYY-MM-DD_uid}                one per person per day; image lives in
Drive
    uid, date, fileId, createdAt (server time)

  restdays/{YYYY-MM-DD}                  rest-day votes for a day
    votes: { uid: server time, … }

  penalties/{autoId}                     log of penalty changes
    penalty, effectiveDate, setBy, setAt

  targets/{autoId}                       log of weekly-target changes
    target, effectiveDate (a Monday), setBy, setAt

  tzchanges/{autoId}                     log of members' timezone changes (phase
5.1)
    uid, timezone, effectiveDate (their next day), setAt

  currencies/{autoId}                    log of pot-currency switches (phase
5.1)
    from, currency, rate (1 from = rate currency), penalty, effectiveDate,
setBy, setAt

private/{uid}/                           only you can read or write anything
```

```
here
  settings/main                          targets { kcal, protein, carbs, fat,
fibre }, weightUnit, goalWeightKg
  meals/{autoId}                         date, meal, name, qty, unit, nutrients,
food (copy for Recent), createdAt
  foods/{autoId}                         your foods & recipes: name, kind,
servingLabel, servingGrams, perServing, barcode?, ingredients?
  weights/{YYYY-MM-DD}                   kg, date
  progress/{autoId}                      date, fileId (in YOUR Google Drive),
note
  settings/training                      goal, daysPerWeek, sessionMin,
experience, restSec, limitations, equipment, customEquipment
  workouts/{autoId}                      date, title, notes, exercises [{ key,
name, muscle, sets [{ reps, kg }] }]
  exercises/{autoId}                     your own exercises: name, muscle, eq
  routines/{autoId}                      name, items [{ key, name, muscle, sets,
reps }]
  measurements/{YYYY-MM-DD}              date, cm { waist, chest, hips, arm,
thigh, neck }, bodyFat
```

Why the IDs look like `2026-09-28_abc123`: using `<date>_<uid>` as the document ID makes "one per person per day" automatic. A second write to the same ID would be an *update*, which the rules forbid.

---

7. Security: who can do what

All enforced by `firestore.rules` on Google's servers (and, for photos, by `Code.gs`).

Thing	Read	Create	Change / delete
Your profile	any signed-in user (name, avatar)	you	you
Passcode lookup	fetch one if you know it; never list all	only the group's creator, with the group	never
Group	members	any signed-in user with a **verified email** (as creator)	members: rename, photo rule only
Members	members	yourself, with a **verified email** and the real passcode, server join time	yourself; join time and timezone are locked; **Leave** once (server time)
Check-ins	members	yourself, while you haven't left, server time, `<date>_<you>`; photo required if the group says so	delete your own within 24 h
Photos	members	yourself, server time, `<date>_<you>`	delete your own within 24 h
Rest votes	members	add/remove **your own** vote, server time	never delete the day
Penalty / target changes	members	any member, server time, valid values	never (permanent log)
Timezone changes	members	**your own** only, server time	never (permanent log)
Currency switches	members	any member, server time, one of ₹ $ £ C$, rate > 0	never (permanent log)

Thing	Read	Create	Change / delete
Photo files (Drive)	members of that group, via Apps Script	members, via Apps Script	only the organiser, in Drive
Everything under `private/{you}/` (meals, foods, targets, weight, progress records, training, workouts, routines, measurements)	**only you**	**only you**	**only you**
Progress photo files	**only you** (they're in your own Drive)	you	you

What stays trust-based: FitPot can prove *when* you checked in and that the photo was taken live in the app. It can't prove you actually trained. That's the friend group's job. Likewise, the exchange rate on a currency switch is editable, and the log shows who set it; agree on it as a group.

8. Glossary

Term	Meaning
Access token	A short-lived key from Google that lets an app act in your account, only within the permission you granted.
Closed-through day	The last day that has ended for every member; the ledger settles up to it.
Exchange rate	How much of one currency equals one unit of another on a given day.
IANA timezone	A timezone's standard name, like `Asia/Kolkata` or `America/New_York`; it knows the daylight-saving rules.
Barcode (EAN/UPC)	The product number printed by the manufacturer; the same in every shop.
OAuth	The standard "Allow this app to…" permission flow.
Scope	Exactly which permission an app asks for (FitPot: `drive.file`).
API	A way for one program to ask another for something (e.g. the app asking Firestore for check-ins).
API key	An identifier for your Firebase project. Not a password; the rules protect the data.
Base64	A way to turn binary data (like a photo) into plain text so it can travel inside a JSON message.
Batch	Several writes that succeed or fail together.
Build	Turning source code into the files browsers run (`dist/`).
Cache	A local copy kept to avoid re-downloading.
Collection / document	Firestore's folders and records.
Commit	A saved snapshot of the code in Git.
Component	A reusable React building block (a button, a screen).
CORS	Browser rule about which websites may call which servers. The app calls Apps Script with a "simple" request to stay within it.
Deploy	Publishing a new version so people can use it.

Term	Meaning
Hook	A React function starting with `use…` that gives a component data or behaviour.
1RM / estimated 1-rep max	The most you could lift once; estimated from any set with the Epley formula.
PR (personal record)	Your best ever on an exercise: heaviest weight, best estimated 1RM, or most reps.
Progressive overload	Doing a little more than last time (reps or weight) so muscles keep adapting.
Routine	A saved list of exercises with target sets × reps, started in one tap.
Volume	Weight × reps, added up over sets; a rough measure of total work.
Email verification	Clicking a link sent to your inbox to prove the address is yours.
Linking	Adding a second way to sign in (e.g. a password) to the same account.
Re-authentication	Signing in again right before a sensitive change, so a borrowed phone can't change your password or delete your account.
ID token	Short-lived signed proof of who you are, issued by Firebase Auth.
JSON	The `{"key": "value"}` text format used to send data around.
Lint	Automatic checking of code for mistakes.
Listener / subscription	A live connection that pushes updates to the app.
Manifest	The PWA file describing the app's name, icon and colours.
Repo	A project's folder on GitHub, with its full history.
Rules	Firestore's server-side permission checks.
Secret	A private value stored in GitHub for the build.
Server timestamp	Time filled in by Google's clock, not the phone's.
Service worker	Background script that caches the app and handles updates.
Spark plan	Firebase's free plan, with no billing account.

Term	Meaning
uid	A user's permanent Firebase ID.
Web app (Apps Script)	A script deployed with a public URL that can receive requests.